

Стандарт разработки на TypeScript

TypeScript файлы автоматически проверяются на соответствие стандарту через [СТАН](#).

Для самоконтроля используйте плагин к линтеру TSLint или ESLint для вашей IDE — [WebStorm](#) или [VS Code](#). Это поможет вам следовать стандартам и избежать типовых ошибок.

Функции, методы, операторы

Конструкторы

В фреймворке Wasaby используются [стандартные классы](#) для TypeScript, которые должны использоваться и для создания классов-наследников.

```
1 class Jedi() {
2     constructor() {
3         console.log('new jedi');
4     }
5
6     fight() {
7         console.log('fighting');
8     }
9
10    block() {
11        console.log('blocking');
12    }
13 }
```

Наследование на чистых прототипах должно использоваться только в крайних случаях (например, из соображений производительности).

- Назначайте метод прототипу вместо замены прототипа на новый объект. Замена прототипа на другой объект делает наследование невозможным.

```
1 function Jedi() {
2     console.log('new jedi');
3 }
4
5 // Плохо
6 Jedi.prototype = {
7     fight() {
8         console.log('fighting');
9     },
10
11    block() {
12        console.log('blocking');
13    }
14 };
15
16 // Хорошо
17 Object.assign(Jedi.prototype, {
18     fight() {
19         console.log('fighting');
20     },
```

```

21
22     block() {
23         console.log('blocking');
24     }
25 });

```

- Методы могут возвращать `this` для формирования цепочек вызовов, если поведение экземпляра класса настраивается через последовательный вызов множества методов.

```

1  // Плохо
2  class Jedi() {
3      constructor() {
4          console.log('new jedi');
5      }
6
7      jump(): void {
8          this.jumping = true;
9      }
10
11     push(): void {
12         this.pushing = true;
13     }
14
15     yell(phrase: string): void {
16         this.yelling = phrase;
17     }
18 }
19
20 const luke = new Jedi();
21 luke.jump();
22 luke.yell('Cowabunga!');
23 luke.push();

```

```

1  // Хорошо
2  class Jedi() {
3      constructor() {
4          console.log('new jedi');
5      }
6
7      jump(): this {
8          this.jumping = true;
9          return this;
10     }
11
12     push(): this {
13         this.pushing = true;
14         return this;
15     }
16
17     yell(phrase: string): this {
18         this.yelling = phrase;
19         return this;
20     }
21 }
22
23 const luke = new Jedi();
24 luke.jump()

```

```
25     .yell('Cowabunga!')
26     .push();
```

- Вы можете написать пользовательский метод `toString()`, возвращающий текстовое представление экземпляра класса, только убедитесь, что он успешно работает и не вызывает никаких побочных эффектов.

```
1  interface IJediOptions {
2      name?: string;
3  }
4
5  class Jedi() {
6      protected _options: IJediOptions;
7
8      constructor(options: IJediOptions) {
9          this._options = options || {};
10     }
11
12     get name(): string {
13         return this._options.name || 'anonymous';
14     }
15
16     toString(): string {
17         return `[Jedi:${this.name}]`;
18     }
19 }
```

Комментарии

1. В начале каждого файла следует указать его назначение, а также ФИО ответственного.
2. В файле с исходным кодом должны быть прокомментированы все классы, опции, методы, события, команды и типы данных, а также события каналов.
3. Для вынужденных решений, когда код осознанно написан "плохо", следует добавить ссылку на Задачу/Ошибку, после решения которой код можно улучшить. В результате разработчик может оперативно понять причину и отследить решение вопроса.
4. Для документирования API используйте синтаксис [JSDoc](#).

FIXME и TODO

Использование префиксов **FIXME** или **TODO** в комментариях помогает другим разработчикам быстро понять, что вы указываете на проблему, к которой нужно вернуться позже, или если вы предлагаете решение проблемы, которое должно быть реализовано. Такие комментарии отличаются тем, что призывают к действию, не описывая текущее поведение.

В комментариях следует добавить ссылку на Задачу/Ошибку, из-за которой принято решение создать именно такой код. В результате разработчик может оперативно понять причину и отследить решение вопроса.

- Используйте `// FIXME`: для аннотирования проблемы:

```
1  function calculator() {
2      // FIXME: здесь не следует использовать глобальную переменную:
3      total = 0;
4
5      return this;
6  }
```

- Используйте `// TODO`: для аннотирования решения проблемы

```
1  function calculator() {
2      // TODO: total должен конфигурироваться параметрами опции
3      this.total = 0;
4  }
```

```
5 |     return this;
6 | }
```

Соглашения об именовании

- Все имена переменных и функций должны быть написаны на латинице и отвечать правилам английского языка. Русские слова в латинской транслитерации не допускаются!
- Константы, хранящие предустановленные значения, должны содержать прописные буквы, цифры и символ подчеркивания.

```
1 | const CONSTANTS_NAMED_LIKE_THIS = 10;
```

Избегайте однобуквенных имен функций. Имена должны давать представление о том, что делает эта функция.

```
1 | // Плохо
2 | function q() {
3 |     // some code
4 | }
1 | // Хорошо
2 | function query() {
3 |     // some code
4 | }
```

- Используйте подчеркивание `_` для именования внутренних методов и переменных объекта

```
1 | // Плохо
2 | class Animal {
3 |     protected __kind: string = '';
4 |     protected kind_: string = '';
5 |
6 |     protected __getMovementType(): string {
7 |     }
8 | }
1 | // Хорошо
2 | class Animal {
3 |     protected _kind: string = '';
4 |
5 |     protected _getMovementType(): string {
6 |     }
7 | }
```

Подключение статических зависимостей

- Для подключения статических зависимостей используйте инструкцию [import](#).

```
1 | import {Control, TemplateFunction} from 'UI/Base';
2 | import {ReactiveObject} from 'Types/entity';
```

- Если подключаемый модуль не является модулем, написанным на языке TypeScript, используйте конструкции [import * as](#).

```
1 | import * as Abstract from 'Core/Abstract';
```

Подключение динамических зависимостей

- Для подключения динамических зависимостей используйте [dynamic import](#).
- Обязательно обрабатывайте ошибки подключения! Как минимум их следует залогировать.

```

1 // Плохо: не обработаны ошибки
2 ...
3 const buttons = await import('Controls/buttons');
4 // Work with buttons
1 // Хорошо
2 import {IoC} from 'Env/Env';
3 ...
4 try {
5     const buttons = await import('Controls/buttons');
6     // Work with buttons
7 } catch (err) {
8     IoC.resolve('ILogger').error('My/Component', err);
9 }
1 // Плохо: не обработаны ошибки
2 ...
3 import('Controls/buttons').then((buttons) => {
4     // Work with buttons
5 });
1 // Хорошо
2 import {IoC} from 'Env/Env';
3 ...
4 import('Controls/buttons').then((buttons) => {
5     // Work with buttons
6 }).catch ((err) => {
7     IoC.resolve('ILogger').error('My/Component', err);
8 });

```

- Для подключения нескольких зависимостей сразу используйте [Promise.all\(\)](#).

```

1 // Хорошо
2 import {IoC} from 'Env/Env';
3 ...
4 Promise.all(['ModuleA', 'ModuleB'].map(
5     (name) => import(name)
6 )).then(([ModuleA, ModuleB]) => {
7     // Work with ModuleA and ModuleB
8 }).catch ((err) => {
9     IoC.resolve('ILogger').error('My/Component', err);
10 });
1 // Хорошо
2 import {IoC} from 'Env/Env';
3 ...
4 try {
5     const [ModuleA, ModuleB] = await Promise.all(['ModuleA',
6     'ModuleB'].map(
7         (name) => import(name)
8     ));
9     // Work with ModuleA and ModuleB
10 } catch (err) {
11     IoC.resolve('ILogger').error('My/Component', err);
12 }

```

- Не используйте глобальные переменные `require` и `requirejs`.
- Не используйте устаревший модуль `Core/moduleStubs`.

```

1 // Плохо: использование глобальной переменной require.
2 ...
3 require(['Controls/buttons'], (buttons) => {
4     // Work with buttons
5 });
1 // Плохо: использование глобальной переменной requirejs
2 ...
3 requirejs(['Controls/buttons'], (buttons) => {
4     // Work with buttons
5 });
1 // Плохо: использование Core/moduleStubs
2 import * as moduleStubs from 'Core/moduleStubs';
3 ...
4 moduleStubs.require(['Controls/buttons'], (result) => {
5     // Work with result
6 });

```

Модули

- Имя файла должно совпадать с именем экспортируемой модулем сущности по умолчанию (класса, функции, константы).
- Если экспортируемая по умолчанию сущность не определена, имя файла должно характеризовать совокупность экспортируемых модулем сущностей.

```

1 // animals/Cat.ts
2 import Abstract from './Abstract';
3
4 export default class Cat extends Abstract {
5     protected _kind: string = 'cats';
6 }
1 // animals.ts
2 import Abstract from './animals/Abstract';
3 import Cat from './animals/Cat';
4
5 export {
6     Abstract,
7     Cat
8 }

```

Интерфейсы

```

1 export interface ICheckableOptions {
2     value?: boolean;
3 }
4
5 /**
6  * Interface for 2-position control.
7  *
8  * @interface Controls/_toggle/interface/ICheckable
9  * @public
10 * @author Красильников А.С.
11 */
12 export interface ICheckable {
13     readonly '[Controls/_toggle/interface/ICheckable]': boolean;

```

```

14 }
15 /**
16  * @name Controls/_toggle/interface/ICheckable#value
17  * @cfg {Boolean} Current state.
18  */
19 /**
20  * @event Controls/_toggle/interface/ICheckable#valueChanged Occurs
  when value changes.
21  * @cfg {Boolean} New value.
22  */

```

1. Файл с интерфейсом экспортирует отдельный интерфейс с опциями. Название – имя интерфейса + Options.

```

1 export interface ICheckableOptions {
2     value?: boolean;
3 }

```

2. Файл с интерфейсом экспортирует сам интерфейс:

```

1 export interface ICheckable {...}

```

3. Вся документацию по интерфейсу опишите в файле интерфейса.

4. Если интерфейс используется не более чем в одном классе, для него можно не создавать отдельный файл.

5. Если интерфейс используется в рамках одной библиотеки, его размещают в директории interface внутри приватной папки библиотеки. И такой интерфейс должен экспортироваться библиотекой.

6. Если интерфейс используется в нескольких библиотеках, его размещают в директории interface в корне интерфейсного модуля. И такой интерфейс должен экспортироваться библиотекой interface.

Это правило используется в интерфейсном модуле [Controls](#). Папка interface необязательная. В корне интерфейсного модуля можно разместить общие для нескольких библиотек интерфейсы.

```

1 Controls/_toggle/interface/ICheckable.d.ts

```

7. В файле контроля, реализующего интерфейс – импортируйте интерфейс и пропишите в директиве implements.

```

1 import ICheckable from './interface/ICheckable';

```

```

1 class BigSeparator extends Control implements ICheckable {...}

```

8. Описание интерфейса в доклетах идет прямо перед его объявлением в коде, а описание опций, методов и событий — в конце файла.

```

1 /**
2  * Interface for 2-position control
3  *
4  * @interface Controls/_toggle/interface/ICheckable
5  * @public
6  * @author Красильников А.С.
7  */
8 export interface ICheckable {
9     readonly '[Controls/_toggle/interface/ICheckable]': boolean;
10 }
11 /**
12  * @name Controls/_toggle/interface/ICheckable#value
13  * @cfg {Boolean} Current state.
14  */

```

```

15  /**
16  * @event Controls/_toggle/interface/ICheckable#valueChanged Occurs
    when value changes.
17  * @cfg {Boolean} New value.
18  */

```

9. Поле типа Boolean с именем интерфейса в квадратных скобках делается по двум причинам:

- В TS/ES6 нет нативной проверки на имплементацию инстансом интерфейса (аналога instanceof для интерфейсов). Подобное поле с уникальным именем позволит писать проверки вида:

```

1  if (instance['[Controls/_toggle/interface/ICheckable]'])

```

- TS ругается на пустой интерфейс.

Интерфейсы для опций

1. Поле _options описано в базовом UI/Base:Control, и его можно не описывать в своих контролах.

2. Интерфейс опций используйте в [хуках жизненного цикла](#).

```

1  protected _beforeMount(newOptions: IBigSeparatorOptions): void {
2      this._iconChangedValue(newOptions.value);
3  }
4
5  protected _beforeUpdate(newOptions: IBigSeparatorOptions): void {
6      this._iconChangedValue(newOptions.value);
7  }

```

Классы и методы

1. Контрол — это класс. Базовый Control – Generic class, который принимает интерфейс опций.

```

1  // Хорошо
2  class BigSeparator extends Control<IBigSeparatorOptions> implements
    ICheckable {...}
1  // Плохо
2  var BigSeparator = Control.extend({...})

```

2. Приватные методы оформляйте синтаксисом TypeScript — название метода начинается с подчеркивания. Например: _iconChangedValue.

```

1  // Хорошо
2  class BigSeparator extends Control<IBigSeparatorOptions> implements
    ICheckable {
3
4  private _iconChangedValue(value: boolean): void {
5      if (value) {
6          this._icon = 'icon-AccordionArrowUp ';
7      } else {
8          this._icon = 'icon-AccordionArrowDown ';
9      }
10 }
11 }
1  // Плохо
2  var protected = {
3  iconChangedValue: function (self, options) {
4      if (options.value) {

```

```

5         this._icon = 'icon-AccordionArrowUp ' ;
6     } else {
7         this._icon = 'icon-AccordionArrowDown ' ;
8     }
9 }
10 }

```

3. Хуки жизненного цикла — protected методы.

```

1 protected _beforeMount(newOptions: ICheckableOptions): void {
2     this._iconChangedValue(newOptions.value);
3 }
4
5 protected _beforeUpdate(newOptions: ICheckableOptions): void {
6     this._iconChangedValue(newOptions.value);
7 }

```

4. Обработчики событий – protected методы.

```

1 protected _clickHandler(): void {
2     this._notify('valueChanged', [!this._options.value])
3 }

```

Для аргумента типа события event используется **SyntheticEvent**:

```

1 import {SyntheticEvent} from 'UICommon/Events';

```

При обработке нативных событий в wasaby в обработчик приходит не нативный объект события, а обертка над ним под названием **SyntheticEvent** - generic класс, который принимает интерфейс нативного события. К нативному объекту события можно обратиться через **event.nativeEvent**:

```

1 import {SyntheticEvent} from 'UICommon/Events';
2 ...
3 protected _onMouseDownHandler(event: SyntheticEvent<MouseEvent>):
  void {
4     if (!this._options.readOnly) {
5         const box = this._children.area.getBoundingClientRect();
6         const ratio = Utils.getRatio(event.nativeEvent.pageX, box.left +
  window.pageXOffset, box.width);
7         this._value = Utils.calcValue(this._options.minValue,
  this._options.maxValue, ratio, this._options.precision);
8         this._setValue(this._value);
9         this._children.dragNDrop.startDragNDrop(this._children.point,
  event);
10    }
11 }

```

5. У методов и переменных указывайте корректные модификаторы.

```

1 protected _icon: string;

1 protected _clickHandler(): void {
2     this._notify('valueChanged', [!this._options.value])
3 }

```

6. Расставляйте типы у полей, аргументов методов и возвращаемых значений методов.

```

1 protected _iconChangedValue(value: boolean): void {
2     if (value) {
3         this._icon = 'icon-AccordionArrowUp ' ;
4     } else {
5         this._icon = 'icon-AccordionArrowDown ' ;

```

```

6   }
7   }
1   protected _icon: string;

```

7. Тему и template оформляйте следующим образом (TemplateFunction импортируем из библиотеки UI/Base):

```

1   import {Control, IControlOptions, TemplateFunction} from 'UI/Base';

1   protected _template: TemplateFunction = checkBoxTemplate;
2   import 'css!Controls/toggle';

```

8. Контроль по умолчанию экспортирует себя.

```

1   // Хорошо
2   export default BigSeparator;

1   // Плохо
2   export = BigSeparator;

```

Исключение: На время переходного этапа, если ваш контроль не является частью библиотеки (например, контроль SwitchableArea) и его могут грузить через require, надо писать export = для совместимости.

9. Полное имя контроля в составе JavaScript библиотеки соответствует форме "МодульСБИС/библиотека:контроль", например: Controls/list:View. В этом примере:

- Controls — это имя [модуля СБИС](#). Он физически хранится [здесь](#), а также распространяется в составе [SBIS SDK](#).
- list — это имя JavaScript-библиотеки.
- View — краткое имя контроля в составе библиотеки.

Экспорт из библиотек

```

1   // Хорошо
2   export {default as BigSeparator} from
   'Controls/_toggle/BigSeparator';

1   // Плохо
2   import BigSeparator = require ('Controls/_toggle/BigSeparator');
3
4   export {
5       BigSeparator
6   }

```

Кастомные типы

Если в публичном методе используется какой-то сложный тип аргумента, который объявляется в вашем модуле, такие типы лучше тоже экспортировать. Они могут потребоваться при задании типов переменных в прикладном коде.

Пример: тип where – аргумент вызова фильтрации на источнике данных

```

1   type Where = IHashMap<string> | ((item: any, index: number) =>
   boolean);

```

Набор утилит

Если модуль экспортирует утилиты: набор методов или констант, каждый метод/константу экспортируем отдельно.

```
1 export const showType = (...);
2
3 export function getMenuItems<T>(items: RecordSet<T> | T[]):
  ChainAbstract<T>{
4     return factory(items).filter((item) =>{
5         return item.get('showType') !== this.showType.TOOLBAR;
6     })
7 }
```

Рекомендации от команды TypeScript

Следуйте [официальным рекомендациям команды TypeScript](#). Ниже приведён перевод этих рекомендаций.

Основные типы

Никогда не используйте типы Number, String, Boolean, Symbol или Object. Эти типы ссылаются на непримитивные упакованные объекты, которые почти никогда не используются по делу в JavaScript коде.

```
1 // Плохо
2 function reverse(s: String): String;
```

Вместо этих типов используйте number, string, boolean, symbol, object.

```
1 // Хорошо
2 function reverse(s: string): string;
```

Обобщения

Никогда не создавайте обобщения, не использующие свой типовой параметр. Подробнее читайте в [FAQ TypeScript](#).

any

Не используйте any. Исключением являются ситуация, когда вы переводите проект с JavaScript на TypeScript. Компилятор считает any как команду отключить проверку типов. Эффект будет сравним с добавлением @ts-ignore около каждого использования переменной. Такое поведение может быть очень полезно в начале миграции с JavaScript на TypeScript, т.к. вы можете пометить как any те вещи, которые ещё не были переведены, но в полноценных TypeScript проектах вы просто отключаете проверку типов для тех частей программы, в которых используется any. Если вы не знаете какой тип принимает ваша функция, или вы хотите принимать любой тип, т.к. вы просто передаёте его дальше, то используйте [unknown](#).

Типы возвращаемых значений коллбеков

Не используйте any в качестве возвращаемого значения коллбека, чьё возвращаемое значение будет проигнорировано:

```
1 // Плохо
2 function fn(x: () => any) {
3     x();
4 }
```

Вместо этого используйте тип void:

```
1 // Хорошо
2 function fn(x: () => void) {
3     x();
4 }
```



Использование `void` более безопасно, потому что это предотвращает случайные использования возвращаемого значения.

```
1 function fn(x: () => void) {
2     var k = x(); // упс! хотели написать что-то другое
3     k.doSomething(); // ошибка, но всё было бы нормально, если бы
    в качестве возвращаемого значения использовался any
4 }
```

Необязательные параметры в коллбеках

Не используйте необязательные параметры в коллбеках, кроме случаев, когда вы в этом действительно уверены:

```
1 // Плохо
2 interface Fetcher {
3     getObject(done: (data: unknown, elapsedTime?: number) => void): void;
4 }
```

Такая запись имеет следующее значение: коллбек `done` может быть вызван либо с одним аргументом, либо с двумя. Автор, скорее всего, хотел показать, что коллбек может не обращать внимания на параметр `elapsedTime`, но для этого не нужно делать параметр необязательным - передача меньшего количества параметров в коллбек всегда допустима.

Вместо этого делайте все параметры коллбеков обязательными:

```
1 // Хорошо
2 interface Fetcher {
3     getObject(done: (data: unknown, elapsedTime: number) => void): void;
4 }
```

Перегрузки и коллбеки

Не создавайте несколько перегрузок, отличающихся только числом параметров у коллбека:

```
1 // Плохо
2 declare function beforeAll(action: () => void, timeout?: number): void;
3 declare function beforeAll(
4     action: (done: DoneFn) => void,
5     timeout?: number
6 ): void;
```

Вместо этого создавайте одну перегрузку с максимальным количеством параметров у коллбека:

```
1 // Хорошо
2 declare function beforeAll(
3     action: (done: DoneFn) => void,
4     timeout?: number
5 ): void;
```



Всегда допустимо передавать в коллбек меньшее количество аргументов. Добавление короткой перегрузки в начало позволяет передавать некорректно-типизированные функции, т.к. они подходят к первой перегрузке.

Порядок перегрузок

Не помещайте более широкие перегрузки перед узкими:

```
1 // Плохо
2 declare function fn(x: unknown): unknown;
3 declare function fn(x: HTMLElement): number;
```

```

4 declare function fn(x: HTMLDivElement): string;
5 var myElem: HTMLDivElement;
6 var x = fn(myElem); // x: unknown

```

Вместо этого располагайте более широкие перегрузки после узких:

```

1 // Хорошо
2 declare function fn(x: HTMLDivElement): string;
3 declare function fn(x: HTMLElement): number;
4 declare function fn(x: unknown): unknown;
5 var myElem: HTMLDivElement;
6 var x = fn(myElem); // x: string

```



TypeScript всегда выбирает первую подходящую перегрузку. Когда широкая перегрузка стоит раньше, чем узкая, узкая перегрузка по сути оказывается скрытой и не может быть вызвана.

Используйте необязательные параметры

Не создавайте несколько перегрузок, которые отличаются только последними параметрами:

```

1 // Плохо
2 interface Example {
3   diff(one: string): number;
4   diff(one: string, two: string): number;
5   diff(one: string, two: string, three: boolean): number;
6 }

```

Вместо этого используйте необязательные параметры:

```

1 // Хорошо
2 interface Example {
3   diff(one: string, two?: string, three?: boolean): number;
4 }

```



Обратите внимание, что это подходит только для тех случаев, где все перегрузки имеют возвращаемое значение одного и того же типа.

Это важно по двум причинам:

1. TypeScript определяет совместимость сигнатур, проверяя может ли одна из перегрузок функции быть вызвана с переданными аргументами, причём лишние аргументы допустимы. Например, в следующем коде TypeScript найдёт ошибку только если функция объявлена с необязательными параметрами:

```

1 function fn(x: (a: string, b: number, c: number) => void) {}
2 var x: Example;
3 // При записи с перегрузками - всё хорошо, используем первую
  перегрузку
4 // При записи с необязательными параметрами - ошибка
5 fn(x.diff);

```

2. Код, использующий "строгую проверку на null". Поскольку пропущенные параметры в JavaScript выглядят как undefined, то обычно допустимо передать undefined явным образом в функцию с необязательными параметрами.

```

1 var x: Example;
2 // При записи с перегрузками упадёт ложная ошибка, потому что
  был передан undefined вместо string
3 // При записи с необязательными параметрами всё хорошо
4 x.diff("something", true ? undefined : "hour");

```

Используйте объединения

Не создавайте перегрузки, отличающиеся типом одного аргумента:

```
1 // Плохо
2 interface Moment {
3   utcOffset(): number;
4   utcOffset(b: number): Moment;
5   utcOffset(b: string): Moment;
6 }
```

Вместо этого используйте объединения, если это возможно:

```
1 // Хорошо
2 interface Moment {
3   utcOffset(): number;
4   utcOffset(b: number | string): Moment;
5 }
```



Обратите внимание, что мы не сделали `b` необязательным параметром, потому что типы возвращаемого значения разные.

Это важно для людей, которые "пробрасывают" значение в вашу функцию:

```
1 // Хорошо
2 function fn(x: string): void;
3 function fn(x: number): void;
4 function fn(x: number | string) {
5   // При записи с перегрузками упадёт ложная ошибка
6   // При записи с объединением всё хорошо
7   return moment().utcOffset(x);
8 }
```

См. также

- [Подготовка окружения для TypeScript](#)
- [Порядок разработки интерфейсного модуля на TypeScript](#)